

COP290 Assignment 2 Report

Task Board: Design Decisions and Rationale

Team Members: 2024CS10924, 2024CS10524

March 22, 2026

1 Objective and Scope

This report documents the key design decisions taken while implementing our Task Board application for COP290 Assignment 2. Our objective was not only to satisfy functional requirements, but also to make decisions that keep the system maintainable, auditable, and robust under realistic usage.

The report focuses on non-obvious choices and trade-offs in:

- authentication and authorization,
- board and workflow modeling,
- issue lifecycle semantics,
- collaboration (comments, mentions, notifications),
- data integrity and recoverability,
- frontend interaction model, and
- testing strategy.

2 Guiding Principles

Before implementation, we adopted the following principles:

1. **Server-side enforcement over UI hints:** all critical constraints (roles, transitions, WIP, membership) are enforced in backend business logic.
2. **Recoverability and auto-healing:** metadata structures that can get out of sync (especially workflow transitions) should self-repair.
3. **Simple data model, expressive behavior:** avoid excessive schema complexity; encode behavior through clear service-layer rules.
4. **Auditability by default:** important state changes should leave durable traces in the database.
5. **Incremental UX performance:** use optimistic UI where safe, followed by backend truth synchronization.

3 Major Design Decisions

3.1 Authentication and Session Management

Decision A1: Access and refresh tokens are both cookie-based (HTTP-only), with refresh token persisted in DB.

- **Why:** aligns with assignment requirement for cookie-based auth and reduces XSS exposure compared to localStorage token handling.
- **Trade-off:** CSRF protection must be considered in production-grade systems (not deeply addressed in assignment scope).

Decision A2: Non-rotating refresh-token model with invalidation on logout.

- **Why:** Piazza confirms both rotating and non-rotating refresh approaches are acceptable; we chose simpler revocation semantics and lower implementation complexity.
- **Trade-off:** rotating tokens can further reduce replay windows.

Decision A3: Bootstrap admin strategy: first registered user becomes Global Admin.

- **Why:** avoids extra seeding steps during local setup and keeps first-run experience smooth.
- **Trade-off:** deterministic seeded admin can be better for controlled deployments.

3.2 Authorization and Role Model

Decision B1: Two-tier RBAC model (Global Role + per-project role).

- **Why:** directly maps assignment semantics: global administration plus project-scoped permissions.
- **Result:** supports the same user having different project roles while preserving system-level authority.

Decision B2: Permission checks centralized into helper utilities.

- **Why:** reduces repeated ad hoc checks and keeps service methods readable.
- **Result:** every service gate uses explicit allowed-role sets (admin/member/viewer combinations) according to operation sensitivity.

Decision B3: Permission matrix (RBAC)

Feature / Action	Global Admin	Project Admin	Member	Viewer
PROJECT LEVEL				
Create Project	✓	✗	✗	✗
Delete Project	✓	✓	✗	✗
Add/Remove Members	✓	✓	✗	✗
Change Member Roles	✓	✓	✗	✗
View Project	✓	✓	✓	✓
BOARD / COLUMN LEVEL				
Create Column (Task/Bug)	✓	✓	✗	✗
Edit Column (Rename/Reorder)	✓	✓	✗	✗
Delete Column	✓	✓	✗	✗
Modify Workflow (Transitions)	✓	✓	✗	✗
View Board	✓	✓	✓	✓
ISSUE (TASK/BUG/STORY)				
Create Issue	✓	✓	✓	✗
Edit Issue Fields	✓	✓	✓	✗
Move Issue Across Columns	✓	✓	✓	✗
Delete Issue	✓	✓	✓	✗
Assign/Reassign Issue	✓	✓	✓	✗
View Issue	✓	✓	✓	✓
COMMENTS / COLLABORATION				
Add Comment	✓	✓	✓	✗
Edit/Delete Own Comment	✓	✓	✓	✗
Delete Any Comment	✓	✓	✗	✗
Mention Users (@username)	✓	✓	✓	✗
View Comment	✓	✓	✓	✓

Decision B4: Global-admin-only administrative surfaces for system-level operations.

- **Project creation:** restricted to Global Admin role in backend service logic and mirrored in frontend visibility.
- **User administration:** a dedicated “Manage Users” page is exposed only to Global Admins, with simple role toggling between USER and ADMIN.
- **Safety rule:** demoting the last remaining Global Admin is blocked at backend level.

3.3 Project, Board, and Column Modeling

Decision C1: Board-local workflow state.

- **Why:** Piazza clarification emphasizes statuses/workflows are per-board.
- **Result:** each board can evolve independently without cross-board coupling.

Decision C2: Column semantics are typed (TODO, INPROGRESS, REVIEW, DONE) and also scoped (STORY vs TASKBUG).

- **Why:** this is one of the most important structural decisions in our system.
- **Non-obvious benefit:** separating **scope** cleanly solves the assignment requirement that stories are not directly movable across regular task columns, while still preserving a status field for stories.

Decision C3: Every board starts with mirrored default columns for both scopes.

- **Why:** avoids null workflow states and guarantees immediate usability.
- **Result:** user can create stories and task/bug issues right away; no manual setup prerequisite.

Decision C4: Story columns are immutable; task/bug columns are customizable.

- **Why:** protects invariant required for story status derivation while still allowing board-specific workflow flexibility for movable issues.

3.4 Workflow Engine Design

Decision D1: Workflow transitions represented by an adjacency matrix persisted per board.

- **Why:** matrix lookup gives constant-time move validation from source and destination indexes.
- **UI mapping:** frontend edits blocked pairs; backend reconstructs matrix.

Decision D2: Default policy is “all transitions allowed”, then selectively blocked.

- **Why:** simpler UX for workflow authoring; admins define exceptions rather than constructing the graph from empty.
- **Trade-off:** restrictive workflows require more explicit blocking, but the model is easier to understand initially.

Decision D3: Automatic workflow self-healing when columns change.

- **Why:** transition metadata can become stale after add/rename/delete/reorder operations.
- **Result:** backend compares tracked column IDs against current board columns and resets matrix when out-of-sync, preventing silent invalid states.

3.5 Issue Model and Story Logic

Decision E1: Issues are polymorphic via enum type (STORY, TASK, BUG) in one table.

- **Why:** supports shared fields (priority, assignee, timestamps) and avoids duplicate logic across separate tables.

Decision E2: Parent-child relation is self-referential; only Story acts as parent by rule.

- **Why:** aligns with clarifications that task/bug should not have their own child tasks.

Decision E3: Stories are blocked from manual drag moves.

- **Why:** assignment explicitly says stories are not directly movable across columns.

Decision E4: Story status is auto-derived from least-progressed child state.

- **Rule:** map child columns to progression levels (TODO < INPROGRESS < REVIEW < DONE), then place story in the minimum level lane.
- **Why:** conservative interpretation that prevents stories from appearing “more complete” than any incomplete child.

- **Non-obvious quality:** this rule naturally guarantees consistency constraints discussed in Piazza.

Decision E5: Deleting a parent issue detaches children (orphan-safe), not cascade-delete.

- **Why:** Piazza accepted both alternatives; we chose information preservation.
- **Trade-off:** requires post-delete cleanup semantics for reporting/UI.

3.6 Lifecycle Timestamps and Movement Semantics

Decision F1: Resolved and closed timestamps are inferred from destination column type.

- **Rule implemented:** entering REVIEW or DONE sets resolvedAt (if not already set); entering DONE sets closedAt.
- **Why:** avoids hardcoding display labels and uses semantic column type metadata.

Decision F2: WIP limits are hard-blocking constraints in backend and pre-checked in frontend.

- **Why:** backend guarantee preserves integrity; frontend pre-check improves user feedback latency.
- **Non-obvious benefit:** dual-layer validation supports both correctness and UX.

3.7 Audit Trail and Activity

Decision G1: Audit log is a first-class table linked to issue and actor.

- **Events covered:** status changes, assignee changes, issue field edits, comment add/edit/delete.
- **Why:** assignment requires storage of audit history even if full dedicated UI is optional.

Decision G2: Activity endpoint provides recent logs with user metadata enrichment.

- **Why:** lightweight task activity feed without exposing raw internal DB structures.

3.8 Comments, Mentions, and Rich Text

Decision H1: Mentions use regex extraction of @username patterns and trigger per-user notifications.

- **Validation:** mentioned users are checked against project membership before notification.
- **Why:** prevents noise and cross-project leakage.

Decision H2: Markdown-like comment rendering implemented manually without external libraries.

- **Supported syntax in renderer:** bold, italic, links, inline code, list items, and mention highlighting.
- **Why:** satisfies assignment constraint of no external rich-text libraries while keeping parser understandable.

3.9 Notification System

Decision I1: Notifications are persistent DB records (not transient UI events).

- **Why:** required by assignment and necessary for historical visibility.

Decision I2: Polling-based frontend retrieval every 2 seconds.

- **Why:** assignment accepts polling; this avoids infrastructure complexity of socket lifecycle.
- **Trade-off:** higher request frequency vs lower implementation complexity.

Decision I3: Optimistic read-state updates in UI.

- **Why:** immediate feedback on mark-read actions while still syncing server state.

3.10 Frontend Interaction and UX Decisions

Decision J1: Native HTML drag-and-drop with optimistic board mutation + server reconciliation.

- **Why:** assignment forbids external drag-drop libraries.
- **Pattern:** update local state instantly, call backend, rollback via re-fetch on failure.

Decision J2: Stories visualized in separate section from tasks/bugs.

- **Why:** directly follows clarifications that stories should not be regular movable cards in kanban lanes.

Decision J3: Priority-aware sorting inside columns.

- **Why:** improves triage visibility and aligns with practical issue-tracking usage.

Decision J4: Dedicated admin management route instead of mixing admin controls into project listing views.

- **Why:** keeps regular project workflow uncluttered while providing a simple and explicit admin-only page for system user-role management.
- **Result:** clearer UX separation between project operations and global administration operations.

3.11 Data Integrity and Schema-Level Choices

Decision K1: DB constraints for uniqueness and indexing are explicit.

- **Examples:** unique username/email; unique project membership pair; indexed foreign keys for issue/project/assignee relationships.
- **Why:** prevents duplicate logical identities and supports query performance under scale.

Decision K2: Unique per-board column names within each scope.

- **Why:** avoids ambiguous status references and simplifies workflow mapping.

Decision K3: Avatar as string payload (compatible with base64 image data).

- **Why:** matches Piazza-approved approach and avoids file-storage infrastructure for assignment scope.

4 Trade-offs and Alternatives We Considered

1. **Refresh token rotation:** stronger security posture but more state management complexity.
2. **Graph edge list vs adjacency matrix:** edge list is sparse-friendly; matrix provides simpler constant-time validation and easier reset logic.
3. **Cascade deletion of story children:** simpler referential cleanup but risks accidental data loss; we chose detachment.
4. **SSE/WebSockets for notifications:** better real-time experience but heavier lifecycle handling; polling was sufficient and accepted.
5. **Single set of columns for all issue types:** less schema nuance but conflicts with story movement constraints; dual-scope columns gave clearer semantics.

5 Testing and Quality Strategy

Decision Q1: End-to-end style backend API tests with sequential state sharing.

- **Why:** validates route wiring, middleware auth behavior, DB interactions, and service logic together.
- **Coverage pattern:** auth start, project flows, dashboard/board/column/issue/comment/notification lifecycle, auth end.

Decision Q2: State object reused across test files for realistic flow continuity.

- **Why:** assignment requires integrated behavior checks beyond isolated unit paths.

6 Conclusion

Our implementation decisions were driven by a balance of assignment compliance, system safety, and practical maintainability. The most significant architectural choice was separating story and task/bug workflow lanes while still keeping a unified issue model; this enabled clean handling of story-specific rules without overcomplicating the schema. Combined with backend-enforced constraints, persistent audit/notification data, and optimistic but recoverable frontend interactions, the project remains robust and explainable under both rubric evaluation and viva-style design questioning.

Repository Links:

<https://github.com/AbhinavPJ/COP290-PROJECT2-BACKEND>

<https://github.com/username1-web/COP290PROJECT2FRONTEND>